# IdeaSecretary - AI Knowledge Organization System

An AI-powered knowledge management system that helps users collect, organize, and retrieve ideas through various input formats (text, images, links, voice).

Project Link: Github

## Features

- 📝 **Multi-format Input**: Text, images, links, and voice
- 🧠 **AI-Powered Organization**: Automatic knowledge categorization and suggestions
- 🌳 **Tree Structure**: Hierarchical knowledge organization
- 📊 **Visualization**:
  - **Tree View** (Default): Expandable/collapsible hierarchical tree display
  - **Mindmap View**: Visual graph representation with SVG rendering
  - Both views support dark mode and responsive design
- 🔍 **Full-Text Search**: Fast query indexing
- 🌍 **Multi-language**: English and Chinese with auto-detection
- 🌓 **Dark Mode**: Comfortable viewing in any lighting condition
- 💾 **Auto-Backup**: Automatic backups every 10 operations
- 📦 **Data Export**: Export knowledge base as ZIP
- 🗺️ **Smart Organization**: Automatic knowledge categorization

## Visualization Features

### Tree View (Default)

- Expandable/collapsible nodes showing parent-child relationships
- Word count and tag display for each idea
- Quick action buttons (edit, delete, add child)
- Smooth animations and intuitive navigation

### Mindmap View

- Visual force-directed graph layout
- Interactive hover tooltips with full idea details
- Zoom and pan controls
- Perfect for exploring knowledge structure

👉 **See TREE_MINDMAP_GUIDE.md for detailed usage instructions**

## Architecture

```
IdeaSecretary/
├── backend/          # Python Flask backend
|   ├── app.py       # Flask application
|   ├── models.py    # SQLAlchemy ORM models
|   ├── config.py    # Configuration
|   ├── routes/      # API endpoints
|   └── services/    # Business logic
└── frontend/        # React frontend
    ├── src/
    |   ├── components/
    |   ├── pages/
    |   ├── services/
    |   ├── i18n/     # Internationalization
    |   └── App.js
    └── package.json
```

## Technology Stack

- **Frontend**: React 18, Tailwind CSS, i18next
- **Backend**: Flask, SQLAlchemy ORM, SQLite
- **AI**: OpenAI API (configurable)
- **Authentication**: JWT tokens with bcrypt

## Quick Start

### Backend Setup

```
cd backend
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
python app.py
```

Backend runs on `http://localhost:5000`

### Frontend Setup

```
cd frontend
npm install
npm start
```

Frontend runs on `http://localhost:3000`

## API Documentation

### Authentication

- `POST /api/auth/register` - Register new user
- `POST /api/auth/login` - Login user
- `GET /api/auth/verify` - Verify JWT token

### Ideas

- `GET /api/ideas/bases` - Get all idea bases
- `GET /api/ideas/<base_id>` - Get idea tree
- `POST /api/ideas/<base_id>/create` - Create idea
- `PUT /api/ideas/<idea_id>` - Update idea
- `DELETE /api/ideas/<idea_id>` - Delete idea

### User

- `GET /api/user/profile` - Get user profile
- `PUT /api/user/settings` - Update settings
- `POST /api/user/openai-config` - Set OpenAI config
- `GET /api/user/openai-config` - Get OpenAI config
- `GET /api/user/statistics` - Get user statistics

### Media

- `POST /api/media/upload` - Upload file
- `GET /api/media/<media_id>` - Download file
- `DELETE /api/media/<media_id>` - Delete file (move to trash)
- `GET /api/media/trash` - Get trash files
- `POST /api/media/trash/cleanup` - Clean up old trash
- `GET /api/media/info` - Get media list

### Search

- `GET /api/search?q=query&base_id=<id>&limit=20` - Search ideas

## Database Schema

### Users

- `id`: UUID

- `username`: Unique username
- `email`: User email
- `password_hash`: Bcrypt hash
- `openai_api_key`: API key
- `openai_api_url`: API endpoint
- `openai_model`: Selected model
- `language`: 'en' or 'zh'
- `dark_mode`: Boolean
- `operation_count`: For backup tracking

### IdeaBases

- Tree structure for knowledge organization
- Stats: idea_count, total_word_count
- Support multiple bases per user

### Ideas

- Hierarchical (parent-child relationships)
- Markdown format content
- AI thoughts/suggestions
- Tags for organization
- Media references

### MediaIndex

- UUID-based file storage
- Trash system with cleanup
- Mime type tracking

### Backups

- Automatic after 10 operations
- User-isolated storage

## Configuration

### Environment Variables

Create `.env` in backend folder:

```
FLASK_ENV=development
FLASK_APP=app.py
SECRET_KEY=your-secret-key
DATABASE_URL=sqlite:///ideasecretary.db
JWT_SECRET_KEY=your-jwt-secret
```

### Frontend

Create `.env.local` in frontend folder:

```
REACT_APP_API_URL=http://localhost:5000/api
```

## Development

### Run Tests

```
# Backend
cd backend
pytest

# Frontend
cd frontend
npm test
```

### Linting

```
# Backend
cd backend
flake8 .

# Frontend
cd frontend
npm run lint
```

## Deployment

### Production Build

```
# Backend
cd backend
gunicorn -w 4 -b 0.0.0.0:5000 app:app

# Frontend
cd frontend
npm run build
```

Deploy frontend build to nginx or CDN.

## Test Users

Demo credentials (to be created using `create_test_data.py` under backend folder):

- Username: `demo`
- Password: `demo123456`
- Email: `demo@ideasecretary.local`

## Future Enhancements

- [ ] Collaboration features
- [ ] Team workspaces
- [ ] Advanced AI features (summarization, clustering)
- [ ] Custom prompts library
- [ ] Integration with external APIs
- [ ] Mobile app
- [ ] Real-time sync

## License

See LICENSE file for details.

## Author

Created by Howl Fang - howl-fang.github.io